

1 Sklearn dan hamma parametrlarni nima vazifa bajarishini ko'ramiz. $\Rightarrow$ va shpalyalka qilib olamiz o'rimiz u-n. (odingda ishlatib ko'ramiz)

HYPERPARAMETER vs PARAMETER

Hyperparametrlar:

- Configuration o'zgaruvchilar $\Rightarrow$ (O'zimiz sozlamalarini beradigan o'zgaruvchi hisoblanadi).
- Berilayotgan Algorithm z ishlashi, qayergacha ishlashi, nimalardan tashkil topishi, ishlatilgan o'zgaruvchi sifatida berishimiz beramiz. Y'ni control (nazorat) biz = qo'limizda bo'ladi.

- Trainingdan oldin qo'yiladi; • Training prosesini nazorat qiladi. (Trainingdan oldin qo'yilyapti ($\text{max_depth} = 10$ va $\text{max_depth} = 50 \Rightarrow$ 1. vaqtga ta'bir qiladi, 2. modelni qayerlarga o'rganishni o'limlaydi) $\Rightarrow$

- Ular datalardan o'rganmaydi (Ammo dataga ta'risi bo'ladi)

Parametrlar:

- Internal coefficient (weights) (Ammo parametrlar datalardan o'rganadi)

- Datadan automatic o'rganadi, va manually kerak emas.

Example: w, b (weights, bias) $\hat{y} = \underset{\text{input}}{w}x + \underset{\text{output parameters}}{b}$

automatic o'rganib
taredi

Hayotiy Analogi

- Params : - Gazni qanchalik bosish
- Terlanish qanday bo'lish k.k
- Hyperparams :
- Gear mode (auto / manual)
- Cruise control speed (o'zimiz speedni belgi-
lashtirish mumkin)

Hyperparameter Tuning

- Model uchun optimal bo'lgan (eng zo'r bo'lgan) hyperparameterlarni izlash process.
- **Nima uchun muhim?**
 - Overfitting bly underfittingni oldini oladi.
 - Modelni aniqlik va chidamlilikni yaxshi laydi.
 - Reduce bias - variance trade off.
 - Saves time and computational resources.
- **Nimaga olib keladi?**
 - Yuqori aniqlik.
 - Low error.
 - Best Generalization.

• Ular ning turlari :

1. Manual Search :

↳ Foydaliligi : Kichik va soddada modellarda past.

↳ Qachon ishlatish : Tez tarjibalar, kam parametrli oddiy modellar.

↳ Hissol : $\text{max-depth} = 3$ yoki $\text{max-depth} = 5$ qiymatini kichik Decision Tree uchun bi-ush.

↳ Afzalligi : Oddiy, ~~hech~~ hech qanday heqiyatlar talab qilinmaydi.

↳ Kamchilik : Samarasiz, eng yaxshi qiymatni otkazib yuborish mumkin.

2. Grid Search :

Grid Search $\Rightarrow$ Hyperparameter tuningni eng mashxar vakillaridan biri hisoblanadi.

• Foydaliligi : Kichik ma'lumot va kam para-
metrklar uchun o'tacha.

• Qachon ishlatish : Kichik parametrlar soni bilan to'liq tekshirish.

• Hissol : Random Forest uchun : $n\text{-estimators} = [50, 100]$ va $\text{max-depth} = [3, 5]$

Afzalligi : Barcha kombinatsiyalar tekshiriladi.

3. Random Search:

- Random Search: $\text{max_depth} = [3, 5, 10]$ shular uchidan **randomly** oladi.
- Foydali ligi: Katta parameter maydoni uchun yuqori.

Qiyoslaganimizda: Manualdan yaxshiroq, ammo Grid Search dan yomonroq.

- Qachon ishlatish: Katta modellar (XGBoost, Deep Learning) bilan

- Misol: **n** - estimators, **max_depth**, **min_samples**, **splitting** taqdimiy kombinatsiyalarini sinash.

- Afzallik: Grid Search'dan tezroq, yaxshi natijalarni tez topadi.

- Kamchilik: Eng optimal qiymatni o'tkazib yuborish mumkin.

4. Bayesian Optimization:

- Foydali ligi: O'zta va katta modellar u-n juda yuqori.

- Qachon ishlatish: Har bir model muhim/ qiymat bo'lgan hollarda (ko'p darajali, Random Forest, neyron tarmoqlar)

- **Hisol**: XGBoost'da "learning_rate", va "n_estimators"ni optimallashtirish.

★ **Taqqoslash**: Grid Search = Bayesian Optimization > Randomly Search. va Manual Search.

★ **Afzalligi**: Samarali, ehtimollik (probability) modellari yordamida optimumni topadi.

- **Kamchiligi**: Kompleks, kutubxonalar talab qiladi (scikit-optimize, Optuna)

5. Optuna / Automated Methods:

- **Foydaliligi**: Katta va murakkab modellar uchun juda yuqori.
- **Daxkon ishlatish**: Complex pipeline, Deep learning.

- **Hisol**: Random Forest uchun n_estimators, max_depth, min_samples_split, min_samples_leaf'ni birga tuning qilish.

★ **Afzalligi**: Kuchli, pruning, multi-objective - optimization, distributed tuning. Imkoniyati.

★ ★ Kamchilik: Kutubxonalar o'rnatish va API ni o'rganish talab qilinadi.

Grid Search : Amaliy

```
import sklearn.model_selection
import GridSearchCV
```

```
# Split dataset
X_train, X_test ...
```

```
# Define parameter grid
param_grid = {
```

```
    "n_estimators": [50, 100, 150],
```

```
    "max_depth": [2, 4, 6]
```

```
}
```

$n_estimators = 4 ta$
 $max_depth = 4 ta$

$4 \times 4 = 16$

```
# Grid Search
```

```
grid_search = GridSearchCV(RandomForestRegressor(), param_grid, cv=3)
```

```
grid_search.fit(X_train, y_train)
```

```
print("Grid Search Best Params: ", grid_search.best_params_)
```

```
# Train final model with the best hyperparameters
```

CV=3 Bu 3-fold cross validation. [DATASET]

3-qismga bo'linadi:

- FOLD 1 } 1. FOLD 1 + FOLD 2 → Train
validation: FOLD 3
- FOLD 2 } 2. Train: FOLD 1 + FOLD 3
validation: FOLD 2
- FOLD 3 } 3. Train: FOLD 1 + FOLD 2
validation: FOLD 3

Keyin shu 3 ta natijani o'rtacha olinadi.

va 3-fold cv
 $16 \times 3 = 48 ta$
 model training bo'ladi

$R^2 = fold1 = 0.72$
 $R^2 = fold2 = 0.75$
 $R^2 = fold3 = 0.79$
 $1/3 = 0.73$

```
rf_grid = RandomForestRegressor(**grid_search.  
best_params_)
```

```
rf_grid.fit(X_train, y_train)
```

```
# Predictions and accuracy
```

```
y_pred_grid = rf_grid.predict(X_test)
```

```
r2_grid = r2_score(y_test, y_pred_grid)
```

```
print("Grid Search Accuracy:" r2_grid)
```

Random Search : Amaliy

• Hammasini tekshirmayman. Tasadifan bir nechta kombinatsiyani tekshiramiz.

```
from sklearn.model_selection import RandomizedSearchCV  
import numpy as np
```

```
# train test split process here  
;
```

```
# Define parameter distribution
```

```
param_dist = {
```

```
    'n_estimators': [50, 100, 150],
```

```
    'max_depth': [2, 4, 6]
```

```
}
```

Bu kombinatsiya: $3 \times 3 = 9$

! deyar bir $n_iter=5$ deb qo'yganmiz.

Sheening ucheen 9 tasiwi hammasi ni emas, faqat 5 ta tasodifiy kombinatsiya tanlanadi.

Masalan:

50	2
100	6
150	4
50	6
150	2

} Boshqa 4 tasi umuman tekshirilmaydi mumkin.

```
random_search = RandomizedSearchCV(
    RandomForestRegressor(),
    param_dist,
    n_iter=5,
    cv=3,
    random_state=42
)
```

$n_iter=5$ Eng muhim joy $\Rightarrow$ Bu 5 ta parameter combination sinab ko'z degani.

Har biri $cv=3$, shee = ucheen : $5 \times 3 = 15$ ta training
 Grid Search'da esa : $9 \times 3 = 27$ bo'lardi.
 $\hookrightarrow$ Demak Random Search tezroq.

```
random_search.fit(x_train, y_train)
print("Random Search params:", random_search.best_params_)
```

Train final model with best hyperparameters

```
rf_random = RandomForestRegressor(random-search
                                   best-params-)
rf_random.fit(X_train, y_train)
```

Predictions and accuracy

```
y_pred_random = rf_random.predict(X_test)
```

```
r2_random = r2_score(y_test, y_pred_random)
print("Random Search Acc R2 score:", r2_random)
```

Random Search absolute best qiymatni topirishga kafolat yo'q.

lekin katta search space'da ko'pincha juda samarali.

Bayesian Optimization.

Use Probability (pip install scikit-optimize)

```
from skopt import BayesianSearchCV
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

Fresh train/test split

⋮

```
# Define search space
```

```
param-space = {
```

```
    'n-estimators': (50, 150) # integer range
```

```
    'max-depth': (2, 6) # integer range
```

```
}
```

```
# Bayesian Search
```

```
bayes_search = BayesSearchCV(RandomForestRegressor(),  
    param-space, n_iter=10, cv=3, random_state=92)
```

```
bayes_search.fit(X_train, y_train)
```

```
print("Bayesian Optimization Best Params:",  
    bayes_search.best_params_)
```

```
# Train final model with best hyperparameters
```

```
rf_bayes = RandomForestRegressor(*** bayes_search.  
    best_params_)
```

```
rf_bayes.fit(X_train, y_train)
```

```
# Predictions and r2 score
```

```
y_pred_bayes = rf_bayes.predict(X_test)
```

```
r2_score_bayes = r2_score(y_test, y_pred_bayes)
```

```
print
```

Oldin sinov natijalaridan foydalanib, keyingi
islatish hyperparameter promising ekanligini aniqlayman
degan.

qo'miniy jarayon:

$$\left. \begin{array}{l} \text{trial } 1 \\ n\text{-estimators} = 70 \\ \text{max-depth} = 3 \\ R^2 = 0.60 \end{array} \right\} \Rightarrow \left\{ \begin{array}{l} \text{trial } 2 \\ n\text{-estimators} = 100 \\ \text{max-depth} = 3 \\ R^2 = 0.73 \end{array} \right.$$

Bayesian model buni ko'rib:

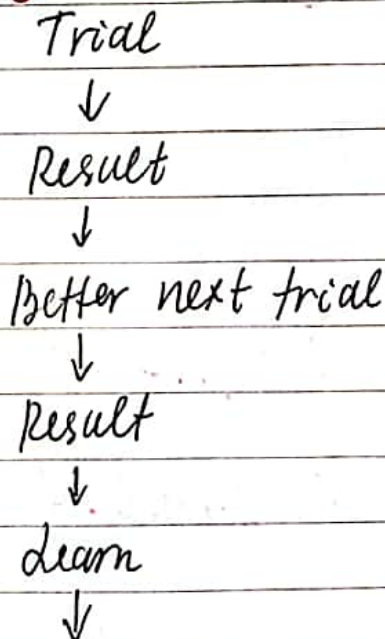
"Aha, hatta n -estimators va max-depth 25' gacha 'ho'layapti'. deb keyingi trialni shu atrofida tanlashi mumkin.

trial 3

$$\left. \begin{array}{l} n\text{-estimators} = 135 \\ \text{max-depth} = 5 \\ R^2 = 0.76 \end{array} \right\} \Rightarrow \text{keyin yana o'zgaradi: trial 4}$$

$$\begin{array}{l} n\text{-est} = 145 \\ \text{max-de} = 6 \\ R^2 = 0.76 \end{array}$$

Yani:



...
Hamashe "Sequential Optimization"

Bayesian Optimization ichki mantiqi

1. Surrogate Model $\Rightarrow$ Bayeslanda yaxshi natija chiqishi mumkinligini taxmin qiladi.
2. Acquisition Function $\Rightarrow$ Keyingi qaysi nuqtani sinash foydalidir?

Bular qismlari, va shu bilan birga strategiya ham mavjud.

Exploration: yangi hudud/ni ko'rish $\Rightarrow$

"Boshqa joyda juda yaxshi parameter bor."

Exploitation: Allaqachon yaxshi topilgan joy atrofiga qarash $\Rightarrow$ "shu hudud yaxshi ekan, yaqinini tekshirib ko'ray."

Bu juda muhim "Concept".

5. OPTUNA

Optuna ham aslida smart hyperparameter optimization framework.

U = ishlatilgan Bayesian Optimizationga yaqin.
Lekin Optuna ancha flexible.

```
def objective(trial):  
    n_estimators = trial.suggest_int(  
        'n_estimators',  
        50,  
        150  
    )
```

```
    max_depth = trial.suggest_int(  
        'max_depth',  
        2,  
        6  
    )
```

```
    clf = RandomForestRegressor(  
        n_estimators = n_estimators,  
        max_depth = max_depth,  
        random_state = 42  
    )
```

```
    return cross_val_score(  
        clf,  
        X_train,  
        y_train,  
        cv = 3  
    ).mean()
```

trial, har bir trial bitta experiment.

Masalan

Optuna tanlaydi:

$n\text{-estimators} = 72 \Rightarrow$ Model esa:

$\text{max-depth} = 3$

$\Rightarrow$ Random Forest Regressor (

$n\text{-estimators} = 72,$

$\text{max-depth} = 3$

)

Cross Validation

fold 1 = 0.65

fold 2 = 0.68

fold 3 = 0.66

} O'rtacha 0.663

Objective function: return 0.663

Optuna buni saylaydi!!!

o
o
o

Shunday heva-latlikda ishlaydi va
belgilanib yaxshirogini oladi.

cross-val-score da esa

$\text{cross-val-score} ($

$\text{clf},$

$X\text{-train}$

$y\text{-train},$

$\text{cv}=3$

)

Bunda RandomForestRegressor

Uchun scoring ko'rsatilmagan

bo'lsa, odatda model = default

score() metrikasi ishlatiladi.

Regressor u-n da R^2

Study yaratish

```
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=10)
```

```
print("Optuna Best Params:", study.best_params)
```

→ uga "maximize" sababi Biz = objective funksiya ni "R2" qaytarayapti. va bu qancha katta bo'lsa shuncha yaxshi.

Ammo, Biz "MSE", "RMSE", va "MAE" ni objective qilsak aksincha:

direction = "minimize" ishlatamiz.

```
study.optimize(
```

```
    objective,
```

```
    n_trials=10
```

```
) # Bu objective() to marta
```

chaqir.

Train final model with best hyperparameters.

```
rf_optuna = RandomForestRegressor(**study.best_params, random_state=42)
```

```
rf_optuna.fit(X_train, y_train)
```

predictions and R2 score

```
y_pred_optuna = rf_optuna.predict(X_test)
```

```
r2_score_optuna = r2_score(y_test, y_pred_optuna)
```

```
print("Optuna r2:", r2_score_optuna)
```